

آزمایشگاه سیستم عامل

پروژه پنجم: مدیریت حافظه در xv6

محمد امین رشید 810102454

سید محمد جواد شریفی 810102469

محمد صادق مهدوی پور 810102519

سوالات تشریحی:

ج 1) هسته سیستم عامل برای پیگیری نگاشت های حافظه یک فرایند از فضای آدرس دهی مجازی استفاده میکند که شامل تعدادی VMA است که هر کدام نقش متفاوتی دارند. مثلاً برای کد برنامه ، استک ، هیپ، دیتا ها و....

سایز یک VMA مضربی از سایز page است (زیرا هر VMA شامل تعدادی page میشود) و هر کدام آدرس شروع و پایانی دارند که این در بخش مجازی به صورت پیوسته است ولی ممکن است در بخش فیزیکی پیوسته نباشد و page ها یک VMA در نواحی مختلفی در فضای فیزیکی قرار گیرند.

هر VMA توسط ساختار vm_area_struct توصیف میشود که شامل همان آدرس شروع و پایان است و هم چنین شامل برخی flag ها برای اینکه مثلاً میتوان روی حافظه چیزی نوشت یا چیزی را خواند.

در xv6 مدیریت حافظه ساده تر است و پیچیدگی کمتری دارد. هر page سایز 4 کیلوبایتی دارد. که در فضای آدرس دهی مجازی به صورت 32 بیتی است (میتواند 4 گیگابایت را آدرس دهی کند) که بخش آدرس های 0 تا 2 گیگابایت شامل کد ، استک ، هیپ است (بخش کاربر) ولی بخش آدرس های 2 گیگابایت تا 4 گیگابایت به فضای فیزیکی مپ میشوند (بخش کرنل). جدایی بخش کاربر و کرنل به دلیل امنیت است تا پراسس های مختلف به حافظه یکدیگر دسترسی نداشته باشند و در بخش کرنل هم با مدیریت سیستم عامل ، مدیریت حافظه انجام میشود.

این مقدار 2 گیگابایت که مرز جدایی را نشان میدهد ، توسط ثابت KERNEL BASE در سیستم عامل define شده است.

هم چنین در xv6 حافظه سلسله مراتبی برقرار است به طوری که 2 سطح دارد که 12 بیت کم ارزش برای افست page است و 2 تا 10 بیت دیگر که page directory و page table را آدرس دهی میکنند.

ج 2) آدرس دهی به صورت سلسله معمولاً به صورت بیش از 2 سطحی است که به این صورت که به جز بیت های سمت راست برای آفست خود یک page ، بقیه بیت ها به جای اینکه همگی صرفاً page number را پیدا کنند ، به چند بخش تقسیم میکنیم (مثلاً 20 بیت را 2 تا 10 بیتی میکنیم) و 10 بیت پر ارزش توسط یک page directory (با سایز خیلی مناسب تر نسبت به حالت 20 بیت) ، به page table مربوطه ارجاع میدهد (یعنی چندین page table ریز تر خواهیم داشت) .

آدرس دهی به صورت سلسله مراتبی منجر به مصرف بهتر حافظه میشود زیرا میدانیم که بیشتر قسمت های فضای آدرس مجازی استفاده نمی شوند و فقط جدول هایی که واقعاً به آدرس های استفاده شده اشاره می کنند، ساخته و در حافظه نگه داشته می شوند و همین باعث میشود که به جای اینکه کل فضای آدرس مجازی (که بسیار بزرگ است) یک جدول بزرگ داشته باشد، فقط مسیرهای واقعا استفاده شده در حافظه نگه داشته شوند.

این کار باعث می شود مصرف حافظه جدول صفحه ها از چند صد مگابایت یا حتی گیگابایت در روش تک سطحی، به چند ده کیلوبایت در روش سلسله مراتبی کاهش پیدا کند که همین باعث میشود که تعداد خطا های page fault کاهش پیدا کند زیرا خود page table در حافظه است و مدیریت بهینه آن ، روی سایر بخش های دسترسی به حافظه تاثیر می گذارد.

حافظه cache هم page table هایی که بیشتر بهشون مراجعه میشود را نگه میدارد (اگر تک سطحی بود امکان نگه داری کل page table نبود) و همین باعث میشود که درصد موفقیت به حافظه برای بار اول زیاد تر شود.

تقریباً همه معماری های مدرن به صورت چند سطحی (سلسله مراتبی) برای ترجمه آدرس مجازی به فیزیکی استفاده می کنند.

ج3 همانطور که از شکل صفحه بعد پیداست سیستم با مدخل (entry) هر دو سطح 32 بیتی 2 بخش اصلی دارد که 20 بیت پر ارزش PPN است (Physical page number) که:

در page table از روی page number آدرس فیزیکی اش را میدهد. در واقع 20 بیت پر ارزش آدرس فیزیکی را میدهد و 12 بیت رتبه پایین با آفست به صورت دقیق تعیین میشود.

این در حالت چند سطحی هم تعمیم داده میشود به صورتی در سطح اول که page directory داریم از روی شماره page directory به ما میگوید که باید به کدام page table مراجعه کنیم در این حالت 12 بیت کم ارزش برابر 0 میگذاریم.

12 بیت کم ارزش که بخش دوم است ، Flags است که :

در اینجا 12 بیت داریم که هر کدام نقش خودشان را دارند و وضعیت چیزی را نشان میدهند:
(1) بیت Present (P) بیت 0 :

اگر 1 باشد: صفحه در حافظه اصلی (RAM) وجود دارد و می توان از آن استفاده کرد.

اگر 0 باشد: صفحه در RAM نیست، در این حالت دسترسی به صفحه باعث page fault می شود.

(2) بیت Writable (W) بیت 1 :

اگر 1 باشد: اجازه نوشتن روی صفحه وجود دارد .

اگر 0 باشد: صفحه فقط خواندنی (ReadOnly) است و نوشتن روی آن باعث خطا می شود.

(3) بیت User (U) بیت 2 :

اگر 1 باشد: صفحه برای برنامه های سطح کاربر (User mode) قابل دسترسی است.

اگر 0 باشد: فقط کدهای سطح هسته (Kernel mode) می توانند به این صفحه دسترسی داشته باشند.

(4) بیت WriteThrough (WT) بیت 3:

اگر 1 باشد: نوشتن ها به صورت WriteThrough انجام می شود (همزمان در کش و حافظه اصلی نوشته می شود).

اگر 0 باشد: معمولاً WriteBack است (ابتدا فقط در کش نوشته می شود و بعداً به حافظه اصلی منتقل می گردد).

(5) بیت Cache Disable (CD) بیت 4 :

اگر 1 باشد: کش برای این صفحه غیرفعال است (همه دسترسی ها مستقیماً با حافظه اصلی انجام می شود).

اگر 0 باشد: کش فعال است و می‌تواند عملکرد را بهبود ببخشد.

(6) بیت Accessed (A) بیت 5 :

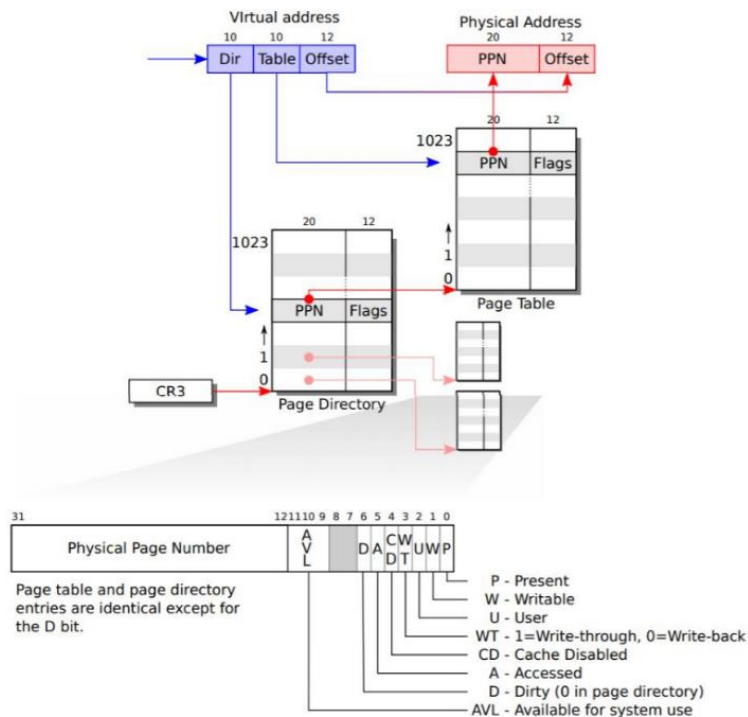
اگر 1 باشد: نشان می‌دهد که از آخرین بار ریست شدن این بیت، حداقل یک بار به این صفحه دسترسی (خواندن یا نوشتن) شده است.

(7) بیت Dirty (D) بیت 6 :

اگر 1 باشد: نشان می‌دهد که از آخرین بار که صفحه تمیز شده، حداقل یک بار روی آن چیزی نوشته شده است. منظور از تمیز شدن این است که محتویات این page نسبت به دیسک یکی باشد.

(8) بیت‌های Available (AVL) و Reserved بیت‌های 7 و 8 به بعد :

این بیت‌ها آزاد هستند و سیستم‌عامل می‌تواند برای اهداف خودش از آنها استفاده کند و در برخی سیستم‌عامل‌ها از این بیت‌ها برای تنظیمات پیشرفته خودش استفاده میکند.



مواردی که بالا تر به عنوان کاربرد هر یک از 12 بیت گفتم ، برای هر دوسطح یعنی هم PTE و هم PDE به صورت کلی صادق است اما در PDE معنا متفاوت است و هر کدام از مواردی که بالا تر تحت عنوان صفحه بیان شد در اینجا به عنوان جدول صفحه بیان میشود . مثلا PRESENT نشان میدهد که PAGE TABLE مربوطه در دیسک هست یا نه .یا مثلا بیت Dirty نشان میدهد که آیا جدول صفحه نسبت به دیسک تغییر کرده

است یا نه. البته در این بخش برخی بیت ها کاربردشان کمرنگ میشود زیرا این سطح به عنوان رابط است تا page table مورد نظر را به ما برگرداند.

ج ۴) تابع `kalloc` وظیفه تخصیص حافظه فیزیکی را بر عهده دارد. این تابع یک صفحه ۴ کیلوبایتی از حافظه آزاد فیزیکی (RAM) را جدا کرده و آدرس ابتدای آن را برمی گرداند. این تابع مستقیماً با سخت افزار RAM در تعامل است و مدیریت لیست صفحات آزاد فیزیکی را انجام می دهد.

ج ۵) تابع `mappages` برای ایجاد mapping در `page table` استفاده می شود. این تابع یک بازه از آدرس های مجازی را به یک بازه از آدرس های فیزیکی `map` می کند. و سطور مورد نیاز `page table` را تنظیم می کند تا پردازنده بتواند آدرس های مجازی را به آدرس های فیزیکی متناظر ترجمه کند. در واقع این تابع هنگام ساختن فضای آدرس یک پردازنده استفاده می شود تا فضای آدرس مجازی برنامه را به حافظه فیزیکی تخصیص یافته توسط `kalloc` پیوند دهد.

ج ۶) خیر لزوماً نه. `proc->sz` فقط اندازه فضای آدرس تخصیص داده شده تا آن نقطه را نشان می دهد؛ اما این تضمین نمی کند که همه صفحات داخل این بازه برای `user` قابل دسترسی باشند. نمونه واضح چنین مسئله ای در `xv6`، صفحه محافظ (`guard page`) است. در هنگام اجرای تابع `exec` برای هر پردازنده، دو صفحه به آن پردازنده اختصاص می یابد. یکی صفحه پشته و دیگری صفحه محافظ (صفحه محافظ برای جلوگیری از سرریز شدن استک استفاده می شود). اما بیت دسترسی کاربر برای صفحه محافظ، غیرفعال می شود. بنابراین در بازه ای که داخل `proc->sz` محسوب می شود، صفحه ای وجود دارد که کاربر حق دسترسی به آن را ندارد.

...

ج ۱۰) در سیستم عامل های واقعی، الگوریتم های تئوری مثل LRU به دلیل سربار بالای به روزرسانی در هر دسترسی به حافظه، استفاده نمی شوند. در عوض، از تقریب های LRU استفاده می شود.

در `Windows`، مدیریت حافظه بر پایه `Working Set` است. سیستم عامل برای هر فرآیند حد بالا و پائینی از صفحات فیزیکی در نظر می گیرد. هنگام فشار بر حافظه، ویندوز با مکانیزم `Working Set Trimming` و با استفاده از الگوریتم `Second-Chance`، صفحاتی را که بیت دسترسی آن ها صفر شده (اخیراً استفاده نشده اند) را از `Working Set` خارج کرده و به لیست صفحات آماده برای آزادسازی منتقل می کند. این کار باعث می شود حافظه فیزیکی به شکلی پویا بین فرآیندها تقسیم شود.

در `Linux`، از یک سیستم `Two-List LRU` (لیست `Active` و `Inactive`) استفاده می شود. صفحات ابتدا در لیست `Active` قرار می گیرند و اگر مدتی به آن ها دسترسی نشود (با بررسی `Access Bit` در جدول صفحات)، به لیست `Inactive` منتقل می شوند. اگر دوباره دسترسی شوند به لیست `Active` برمی گردند. در زمان کمبود

حافظه، سیستم فقط صفحات لیست Inactive را تخلیه می کند. این روش باعث می شود که حافظه پنهان فایل ها (Page Cache) و حافظه برنامه ها به شکلی عادلانه و با کمترین سربار پردازشی مدیریت شوند. این رویکرد، تعادلی بین دقت LRU و سرعت سیستم ایجاد می کند.

بخش عملی:

ابتدا ساختار های گلوبال زیر را تعریف میکنیم که یک page table داریم که یک قفل چرخشی دارد و آرایه از page entry ها که هر کدام میتواند یک page مجازی را به یک آدرس فیزیکی مپ کند. این entry شامل آدرس فیزیکی ، شماره صفحه مجازی (همان آدرس ورودی). Pid پردازش مالک این صفحه و بیت اعتبار که در ابتدا 0 است و هر موقع که صفحه ای اختصاص داده میشود ، این بیت 1 میشود.

بقیه متغیر ها برای الگوریتم های جایگزینی صفحات هستند که از اسمشان مشخص است. برای creation time و last access باید تایم وارد شدن صفحه و تایم آخرین دسترسی به صفحه (خواندن یا نوشتن) ذخیره شود. در اینجا میتوانیم از ticks استفاده کنیم ولی مشکلی که دارد دقت کافی را ندارد و ممکن است تایم 2 create تا page یکی شود برای همین از متغیر global page time استفاده کردم تا هنگام هر read و write در ابتدا یکی زیاد میشود.

```
12
13 int algo_type = 2;
14 int global_page_time = 0;
15
16 struct PageEntry
17 {
18     int vpn;
19     char *paddr;
20     int pid;
21     int valid;
22
23     int creation_time;
24     int last_access;
25     int access_count;
26     int reference_bit;
27 };
28
29 struct
30 {
31     struct spinlock lock;
32     struct PageEntry pages[MAX_PAGES];
33
34     int hit_count;
35     int miss_count;
36 } page_table;
37
```

در سیستم کال write page فرض میکنیم که 2 تا ورودی می گیرد یکی آدرس page مجازی و یک عدد صحیح که قصد ذخیره در آدرس فیزیکی مربوطه داریم. ابتدا با شماره page در entry ها دنبال این page میگردیم (حواسمان هست که باید valid باشد و pid مالک همان pid پردازنده کنونی باشد)

اکنون یا پیدا میشود (hit) یا miss میشود. اگر پیدا شد، مقادیر مربوطه مثل access_count و last_accesss و... آپدیت میشود و در آخر، روی آدرس فیزیکی، مقدار integer مورد نظر را ذخیره میکنیم. اگر پیدا نشد، چک میکنیم که صفحه ای آیا خالی هست؟ اگر valid=0 باشد یعنی خالیه و میتوانیم صفحه را در همان جا قرار دهیم اما اگر همه valid ها 1 بود باید یک page را قربانی کنیم که از طریق تابع get_victim_index() ایندکس قربانی را پیدا می کنیم (از طریق یکی از 4 الگوریتم مختلف). سپس صفحه قربانی را به کمک تابع write_back_victim()، از فریم kalloc شده به صفحه واقعی پردازنده (در نقش دیسک) انتقال می دهیم. در ادامه به کمک تابع load_page_into_frame()، صفحه جدید را به جدول انتقال می دهیم و داده را در آن می نویسیم. اگر آدرس فیزیکی در آن page مقدار دهی نشده باشد یعنی هنوز حافظه ای اختصاص ندادیم و با استفاده از kalloc() آدرس فیزیکی به آن اختصاص میدهیم. الان page جدید در page table ایجاد شد و میتوانیم مقدار دهی اولیه را برای این entry انجام دهیم.

توجه داریم که باید در ابتدای تابع قفل را بگیریم و در آخر قفل را رها کنیم تا از race condition در ساختار page table جلوگیری کنیم.

در سیستم کال read page هم فقط مقدار آدرس صفحه مجازی را دریافت میکنیم و باید data موجود در آدرس فیزیکی را برگردانیم. در صورتی که صفحه مورد نظر پیدا نشد، مطابق قسمت قبل عمل می کنیم. از طریق تابع get_victim_index() ایندکس قربانی را پیدا می کنیم. سپس صفحه قربانی را به کمک تابع write_back_victim()، به صفحه واقعی پردازنده انتقال می دهیم. در ادامه صفحه مورد نظر را به کمک تابع load_page_into_frame()، از صفحه پردازنده به جدول انتقال می دهیم و داده را از آن می خوانیم.

توجه داریم که read page هم مثل write page نوعی access به حافظه محسوب میشود و باید در صورت دسترسی موفق، متغیرهای مربوطه اش آپدیت شود و همینطور ++hit شود و اگر پیدا نشود --miss شود.


```

int sys_write_page(void)
{
    int va, data;
    global_page_time++;

    if (argint(0, &va) < 0 || argint(1, &data) < 0)
    {
        return -1;
    }

    int vpn = (PGROUNDDOWN(va) / PGSIZE);
    int offset = va % PGSIZE;
    int pid = myproc()->pid;

    acquire(&page_table.lock);

    int idx = find_page_entry(vpn, pid);

    if (idx != -1)
    {
        page_table.hit_count++;
        update_access_metadata(idx);
        *(int *) (page_table.pages[idx].paddr + offset) = data;
    }

```

```

    else
    {
        page_table.miss_count++;

        int target = get_target_frame();

        if (page_table.pages[target].valid)
        {
            write_back_victim(target);
        }

        if (page_table.pages[target].paddr == 0)
        {
            page_table.pages[target].paddr = kalloc();
        }

        load_page_into_frame(target, vpn, pid, page_table.pages[target].paddr);

        *(int *) (page_table.pages[target].paddr + offset) = data;
    }

    release(&page_table.lock);
    return 0;
}

```

```

int sys_read_page(void)
{
    int va;
    global_page_time++;

    if (argint(0, &va) < 0)
    {
        return -1;
    }

    int vpn = (PGROUNDDOWN(va) / PGSIZE);
    int offset = va % PGSIZE;
    int pid = myproc()->pid;
    int data = -1;

    acquire(&page_table.lock);

    int idx = find_page_entry(vpn, pid);

    if (idx != -1)
    {
        page_table.hit_count++;
        update_access_metadata(idx);
        data = *(int *) (page_table.pages[idx].paddr + offset);
    }

    else
    {
        page_table.miss_count++;

        int target = get_target_frame();

        if (page_table.pages[target].valid)
        {
            write_back_victim(target);
        }

        if (page_table.pages[target].paddr == 0)
        {
            page_table.pages[target].paddr = kalloc();
        }

        load_page_into_frame(target, vpn, pid, page_table.pages[target].paddr);

        data = *(int *) (page_table.pages[target].paddr + offset);
    }

    release(&page_table.lock);

    return data;
}

```

```

232
233 int find_page_entry(int vpn, int pid)
234 {
235     for (int i = 0; i < MAX_PAGES; i++)
236         if (page_table.pages[i].valid &&
237             page_table.pages[i].vpn == vpn &&
238             page_table.pages[i].pid == pid)
239             {
240                 return i;
241             }
242     return -1;
243 }
244
245 int find_free_frame()
246 {
247     for (int i = 0; i < MAX_PAGES; i++)
248         if (!page_table.pages[i].valid)
249             {
250                 return i;
251             }
252     return -1;
253 }
254
255

```

```

123
124 extern pte_t *walkpgdir(pde_t *pgdir, const void *va, int alloc);
125
126 struct proc* find_proc_by_pid(int pid)
127 {
128     struct proc *p;
129
130     acquire(&ptable.lock);
131     for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
132         if(p->pid == pid && p->state != UNUSED){
133             release(&ptable.lock);
134             return p;
135         }
136     }
137     release(&ptable.lock);
138     return 0;
139 }
140

```

```

82
83 void update_access_metadata(int idx)
84 {
85     page_table.pages[idx].last_access = global_page_time;
86     page_table.pages[idx].access_count++;
87     page_table.pages[idx].reference_bit = 1;
88 }
89
90 int get_target_frame()
91 {
92     int free_idx = find_free_frame();
93     if (free_idx != -1)
94     {
95         return free_idx;
96     }
97
98     return get_victim_index();
99 }
100

```

در زیر، توابع `write_back_victim()` و `load_page_into_frame()` آورده شده است:

```

void write_back_victim(int idx)
{
    struct PageEntry *vic = &page_table.pages[idx];

    if (!vic->valid || vic->paddr == 0)
        return;

    struct proc *p = find_proc_by_pid(vic->pid);
    if (p == 0 || p->pgdir == 0) {
        vic->valid = 0;
        return;
    }

    uint virtual_address = (uint)vic->vpn * PGSIZE;
    pte_t *pte = walkpgdir(p->pgdir, (void*)virtual_address, 0);
    if (pte && (*pte & PTE_P)) {
        memmove(P2V(PTE_ADDR(*pte)), vic->paddr, PGSIZE);
    }

    vic->valid = 0;
}

```

```

void load_page_into_frame(int idx, int vpn, int pid, char *phys_addr)
{
    struct proc *p = find_proc_by_pid(pid);

    if (p && p->pgdir) {
        uint va = (uint)vpn * PGSIZE;
        pte_t *pte = walkpgdir(p->pgdir, (void*)va, 0);

        if (pte && (*pte & PTE_P)) {
            memmove(phys_addr, P2V(PTE_ADDR(*pte)), PGSIZE);
        } else {
            memset(phys_addr, 0, PGSIZE);
        }
    } else {
        memset(phys_addr, 0, PGSIZE);
    }

    page_table.pages[idx].vpn = vpn;
    page_table.pages[idx].pid = pid;
    page_table.pages[idx].valid = 1;

    page_table.pages[idx].creation_time = global_page_time;
    page_table.pages[idx].last_access = global_page_time;
    page_table.pages[idx].access_count = 1;
    page_table.pages[idx].reference_bit = 1;
}

```

در تابع `write_back_victim()` در ابتدا صفحه قربانی را در `vic` و پردازه‌ای که این صفحه متعلق به آن است را در `p` ذخیره می‌کنیم. سپس با استفاده از `walkpgdir` آدرس فیزیکی صفحه واقعی پردازه را در `pte` ذخیره می‌کنیم و بعد با استفاده از تابع `memmove`، داده را از فریم پردازه به صفحه فیزیکی واقعی پردازه انتقال می‌دهیم.

در تابع `load_page_into_frame()` در ابتدا پردازه‌ای که این صفحه متعلق به آن است را در `p` ذخیره می‌کنیم. سپس آدرس فیزیکی صفحه واقعی پردازه را در `pte` ذخیره می‌کنیم. سپس اگر این صفحه وجود داشت، آن را به فریم پردازه منتقل می‌کنیم و در غیر این صورت، فریم را صفر می‌کنیم. در انتهای این تابع نیز فیلدهای این فریم جدید را مقداردهی می‌کنیم.

در آخر طبق صورت پروژه اگر پردازه‌ای خواست `exit` کند، باید بیت `valid` صفحات خودش در `page table` را 0 کند. برای همین از تابع `invalidate page for pid` استفاده می‌کنیم تا به ازای هر `entry` که صاحبش

همین پردازش است ، بیت valid اش را 0 کند. این تابع را در `exit` در فایل `proc.c` فراخوانی میکنیم و تعریف این تابع را در `defs.h` میگذاریم تا در اینجا استفاده کنیم.

```
33 void exit(void)
34 {
35     struct proc *curproc = myproc();
36     struct proc *p;
37     int fd;
38
39     if (curproc == initproc)
40         panic("init exiting");
41
42     // Close all open files.
43     for (fd = 0; fd < NOFILE; fd++)
44     {
45         if (curproc->ofile[fd])
46         {
47             fileclose(curproc->ofile[fd]);
48             curproc->ofile[fd] = 0;
49         }
50     }
51
52     begin_op();
53     iput(curproc->cwd);
54     end_op();
55     curproc->cwd = 0;
56
57     invalidate_pages_for_pid(curproc->pid);
58     acquire(&ptable.lock);
59
60     // Parent might be sleeping in wait().
61     wakeup1(curproc->parent);
62
63     // Pass abandoned children to init.
64     for (p = ptable.proc; p < &ptable.proc[NPROC]; p++)
65     {
66         if (p->parent == curproc)
67         {
68             p->parent = initproc;
69             if (p->state == ZOMBIE)
70                 wakeup1(initproc);
71         }
72     }
73 }
```

```
400
401 void invalidate_pages_for_pid(int pid)
402 {
403     acquire(&page_table.lock);
404     for (int i = 0; i < MAX_PAGES; i++)
405     {
406         if (page_table.pages[i].valid && page_table.pages[i].pid == pid)
407         {
408             page_table.pages[i].valid = 0;
409         }
410     }
411     release(&page_table.lock);
412 }
```

```

int get_victim_index()
{
    int victim = 0;

    if (algo_type == 0) // FIFO
    {
        int min_time = 0x7FFFFFFF;
        for (int i = 0; i < MAX_PAGES; i++)
        {
            if (page_table.pages[i].valid &&
                page_table.pages[i].creation_time < min_time)
            {
                min_time = page_table.pages[i].creation_time;
                victim = i;
            }
        }
    }
    else if (algo_type == 1) // LRU
    {
        int min_access = 0x7FFFFFFF;
        for (int i = 0; i < MAX_PAGES; i++)
        {
            if (page_table.pages[i].valid &&
                page_table.pages[i].last_access < min_access)
            {
                min_access = page_table.pages[i].last_access;
                victim = i;
            }
        }
    }
}

```

```

    else if (algo_type == 2) // LFU
    {
        int min_count = 0x7FFFFFFF;
        for (int i = 0; i < MAX_PAGES; i++)
        {
            if (page_table.pages[i].valid &&
                page_table.pages[i].access_count < min_count)
            {
                min_count = page_table.pages[i].access_count;
                victim = i;
            }
        }
    }
    else if (algo_type == 3) // Second-Chance
    {
        int visited_frame = 0;
        while (visited_frame < 2 * MAX_PAGES) {
            int i = clock_hand;
            clock_hand = (clock_hand + 1) % MAX_PAGES;
            visited_frame++;

            if (!page_table.pages[i].valid)
                continue;

            if (page_table.pages[i].reference_bit == 0) {
                victim = i;
                break;
            }
            page_table.pages[i].reference_bit = 0;
        }
    }

    printf("[EVICT] idx=%d pid=%d vpn=%d\n", victim,
        page_table.pages[victim].pid,
        page_table.pages[victim].vpn);

    return victim;
}

```

الگوریتم های زمان بندی بالا را می نویسیم (FIFO و LRU و LFU و Second-Chance). در ۳ الگوریتم اول به ترتیب کمترین زمان ورود صفحه به page table ، دیر ترین دسترسی به یک صفحه ، کمترین تعداد access به آن صفحه را محاسبه میکنیم.

در الگوریتم second_chance از یک متغیر گلوبال به نام clock_hand استفاده می کنیم که ایندکس فریمی که باید بررسی کنیم را نشان می دهد. در این الگوریتم روی فریم ها حرکت می کنیم تا به اولین فریمی که reference_bit آن صفر است برسیم؛ و تا رسیدن به این فریم، reference_bit تمام فریم هایی که بازدید کرده ایم را از یک به صفر تغییر می دهیم. (به این ترتیب شانس مجدد می دهیم).

(reference_bit برای تمام فریم ها در ابتدا صفر است و به ازای هر read یا write به یک تبدیل می شود).
اینکه از کدام الگوریتم استفاده شود را از طریق متغیر algo_type مشخص میکنیم.

برای تست برنامه سطح کاربر زیر را می نویسیم:

```
int align_up(int x, int a)
{
    int r = x % a;
    if(r == 0)
        return x;
    return x + (a - r);
}

int main()
{
    printf(1, "\n");

    int i;

    char *raw = sbrk(7 * PAGE_SZ);
    if(raw == (char*)-1){
        printf(1, "sbrk failed\n");
        exit();
    }
    int base = align_up(raw, PAGE_SZ);

    for (i = 0; i < PAGES; i++)
    {
        int va = (base + i * PAGE_SZ);
        write_page(va, i * 10);
    }

    for (i = 0; i < PAGES; i++)
    {
        int va = (base + i * PAGE_SZ);
        int value = read_page(va);
        printf(1, "Read from page %d (vpn=%x) => value=%d\n", i, va / PAGE_SZ, value);
    }

    read_page((base + 3 * PAGE_SZ));
    read_page((base + 3 * PAGE_SZ));
    read_page((base + 3 * PAGE_SZ));
    read_page((base + 1 * PAGE_SZ));
    read_page((base + 1 * PAGE_SZ));
    read_page((base + 2 * PAGE_SZ));
    read_page((base + 0 * PAGE_SZ));
    read_page((base + 0 * PAGE_SZ));

    write_page((base + 4 * PAGE_SZ), 4 * 10);
    write_page((base + 5 * PAGE_SZ), 5 * 10);

    exit();
}
```


در این برنامه از تابع کمکی `align_up` استفاده می‌کنیم. این تابع اولین مضرب `a` که بزرگتر یا مساوی `x` است را برمیگرداند.

در ابتدای برنامه به کمک تابع `sbrk` یک حافظه در هیپ رزرو می‌کنیم تا صفحات مجازی برنامه کاربر در آنجا تشکیل شوند (هیپ امن‌تر از حافظه‌های دیگر است). اما تابع `sbrk` لزوماً یک آدرس مضرب `PAGE_SZ` برنمیگرداند. به همین جهت از تابع `align_up` استفاده می‌کنیم.

سپس روی صفحات مجازی 0 تا 3 مقادیر 0 و 10 و 20 و 30 را مینویسیم و سپس این مقادیر را خوانده و چاپ می‌کنیم. (برای صحت سنجی عملکرد توابع `read` و `write`)

سپس چند `read_page` انجام می‌دهیم تا تعداد دسترسی به صفحات را تغییر دهیم و در انتها در صفحات 4 و 5 می‌نویسیم. این دو صفحه در `page table` نیستند و بنابراین با `page fault` مواجه می‌شویم و باید مطابق با سیاست اتخاذ شده، دو صفحه را قربانی کنیم.

در الگوریتم FIFO باید به ترتیب صفحات 0 و 1 قربانی شوند. زیرا زودتر از بقیه ایجاد شده‌اند.

در الگوریتم LRU باید به ترتیب صفحات 3 و 1 قربانی شوند. زیرا 3 و بعد 1 دیرترین استفاده را دارند.

در الگوریتم LFU باید در ابتدا صفحه 2 قربانی شود. زیرا کمترین استفاده را داشته است. سپس صفحه 4 (که مقدار `vpn` آن 7 است) در مکان صفحه 2 قرار می‌گیرد (ایندکس 2 جدول). در اینجا، این صفحه جدید کمترین استفاده را داشته است. (فقط یک بار) پس اکنون همین صفحه باید قربانی شود. بنابراین صفحه 4 حذف می‌شود. یعنی مجدداً ایندکس 2 جدول حذف می‌شود.

در الگوریتم Second-Chance باید به ترتیب صفحات 0 و 1 قربانی شوند. زیرا در ابتدا وقتی همه صفحات را `write` کردیم، `reference_bit` همه آنها 1 شده است. پس در دور اول پیمایش جدول، همه آنها شانس مجدد می‌گیرند و `reference_bit` همگی 0 می‌شود. سپس در دور دوم در ابتدا صفحه 0 حذف می‌شود و صفحه 4 با `reference_bit = 1` جای آن می‌نشیند و سپس صفحه 1 حذف می‌شود و صفحه 5 با `reference_bit = 1` به جای آن قرار می‌گیرد.

خروجی برای هر الگوریتم را می‌بینیم:

(یک `cprintf` در تابع `get_victim_index` قرار داده‌ایم تا مشخصات صفحه‌ای که قربانی می‌شود را چاپ کند.)

:FIFO -۱

```
$ test_page
Read from page 0 (vpn=3) => value=0
Read from page 1 (vpn=4) => value=10
Read from page 2 (vpn=5) => value=20
Read from page 3 (vpn=6) => value=30
[EVICT] idx=0 pid=3 vpn=3
[EVICT] idx=1 pid=3 vpn=4
```

:LRU -۲

```
$ test_page
Read from page 0 (vpn=3) => value=0
Read from page 1 (vpn=4) => value=10
Read from page 2 (vpn=5) => value=20
Read from page 3 (vpn=6) => value=30
[EVICT] idx=3 pid=3 vpn=6
[EVICT] idx=1 pid=3 vpn=4
```

:LFU -۳

```
$ test_page
Read from page 0 (vpn=3) => value=0
Read from page 1 (vpn=4) => value=10
Read from page 2 (vpn=5) => value=20
Read from page 3 (vpn=6) => value=30
[EVICT] idx=2 pid=3 vpn=5
[EVICT] idx=2 pid=3 vpn=7
```

:Second_Chance -۴

```
$ test_page
Read from page 0 (vpn=3) => value=0
Read from page 1 (vpn=4) => value=10
Read from page 2 (vpn=5) => value=20
Read from page 3 (vpn=6) => value=30
[EVICT] idx=0 pid=3 vpn=3
[EVICT] idx=1 pid=3 vpn=4
```